

Express JS @ Glance

Topic	Syntax
Basic Requirement for Express	
Import Express	<code>const express = require('express');</code>
Initialize App	<code>const app = express();</code>
Two Method for route	
GET Route	<code>app.get('/route path', (req, res) => { ... });</code>
POST Route	<code>app.post('/route path', (req, res) => { ... });</code>
Listen to port	
Start Server	<code>app.listen(port no, () => { ... });</code>
Response methods	
Set Header	<code>res.set('Content-Type', 'text/html');</code> or <code>res.type("text/html")</code>
<code>res.write()</code>	<code>res.write('Some content');</code>
Send Response (Text)	<code>res.send('Hello World!'); // Must blank if use res.write in same route.</code> Example <pre>app.get("/", (req,res)=> { res.set ("content-type","text/plain"); res.send ("<h1>Hello</h1>"); });</pre> Or <pre>app.get ("/about", (req,res)=> { res.set ("content-type","text/html"); res.write ("Hello"); res.send (); });</pre> • If you pass in a string, it sets the Content-Type header to text/html.

Topic	Syntax
	If you pass in an object or an array, it sets the application/json Content-Type header, and parses that parameter into JSON.
Send JSON	<code>res.json({ key: 'value' });</code> // Also close connection
<code>res.sendFile()</code>	<code>res.sendFile(__dirname + '/index.html');</code>
Redirect	<code>res.redirect('/new-url');</code> will redirect to specific path

Route Parameters

Route Parameters	<pre>app.get('/calendar/:day/event/:ename', (req, res) => { });</pre> To access object mention req.params url: localhost:portno/calendar/Monday/event/birthday o/p: {"day":"monday","ename":"birthday"} for specific value mention req.params.keyname
------------------	--

Middleware

Type	Syntax	Application	Why it is used	When to use
Built – in middleware	<code>express.json()</code>	JSON	Read JSON body	APIs
	<code>express.urlencoded()</code>	URL Encoded	Read form data	Forms
	<code>express.static()</code>	Static Files	Serve files	Websites
Third party middleware	<code>cookie-parser()</code>	Cookies	Read cookies	Auth, Sessions
	<code>express-session()</code>	Sessions	Store user session	Login systems
	<code>multer()</code>	File upload	Handle file uploads	Forms with files
	<code>cors()</code>	CORS [Cross Origin Resource Sharing]	Allow cross-origin	Allow Frontend to call Backend
Custom middleware	<code>(req, res, next) function</code>	Custom	Custom logic	Logging, Authentication

Middleware Use	<code>app.use((req, res, next) => { ... next(); });</code>
Chained Middleware	<pre>app.use('/route', cb1, cb2, cb3);</pre> cb1,cb2,cb3 are functions which handled in sequence. Mention next() in cb1, cb2 and mention res.send() in cb3

Topic	Syntax		
	Or <pre>app.use('/route', cb1.cb2) app.get('/route', (req,res)=>{... res.send() })</pre>		
Serve Static Files			
Serve Static Files	<pre>app.use(express.static('./')); // incase index.html</pre> or <pre>app.use(express.static('./',{index: “filename”})); // any other name like: form.html or login.html</pre> Note: use this method for all folders which require to serve statically.		
Serve Static (After requiring path module use method path.join())	<pre>app.use(express.static(path.join(__dirname, 'folder name')));</pre>		
Folder structure			
All Files are in One Folder Express -----1.js -----index.html -----1.css -----1.png	Industry Approach Express -----src -----1.js -----public -----index.html -----1.css -----1.png	All Logic files are in src folder Express -----src -----1.js -----index.html -----1.css -----1.png	Non Logic files are in public folder Express -----1.js -----public -----index.html -----1.css -----1.png
Code			
<pre>app.use (expr.static ('./'))</pre> Using “__dirname” <pre>app.use (expr.static (__dirname))</pre>	<pre>app.use (express.static('../public'))</pre> Here we can also join the path <pre>const sp = path.join (__dirname ,'../public'); app.use (expr.static(sp));</pre>	<pre>app.use (express.static("../"))</pre> Here we can also join the path <pre>const sp = path.join (__dirname , "../"); app.use(expr.static(sp));</pre>	<pre>app.use (expr.static("public"))</pre>

Topic	Syntax
Data fetch from Get/Post	
Use get method Form “method= get”	<pre>app.get('/route', (req, res) => { ... });</pre> To access object use req.query Specific values can be fetched by req.query.keyname
Use post method Form “method=post, action= “/data”	<pre>app.use(express.urlencoded({ extended: true }));</pre> <pre>app.post('/route', (req, res) => { ... });</pre> To access object use req.body Specific values can be fetched by req.body.keyname Note: Use app.post on same route mention in form's action. For different route use app.get then, <pre>app.post("/data", (req, res) => { ... })</pre> //then for any other route mention app.get(..) <pre>app.get("/aboutdata", (req, res) => { ... })</pre>
Handle 404	<pre>app.use((req, res) => { res.status(404).send('Not Found');});</pre>
app.get('/'):Handles only GET requests to the exact / path. Example: Only runs for http://localhost:3000/. app.use('/'):Handles all HTTP methods for all paths starting with /. Example: Runs for /, /contact, /about, /anything.	
Cookie	
Cookie Parser	<pre>const cp=require("cookie-parser"); app.use(cp());</pre>

Topic	Syntax
Set Cookie	<pre>res.cookie('name', 'value', option);</pre> option*: {maxAge:2000}; After 2000 ms cookie will expire or <pre>{expires:new Date(Date.now()+10000)}); Expires after 10000 ms from the time it is set.</pre>
Clear Cookie	<pre>res.clearCookie('name');</pre>
Read Cookie	<pre>req.cookies.name_of_cookie;</pre>
Session	
Session Setup	<pre>const sess=require("express-session"); app.use(sess({ secret: 'key', resave: false, //optional saveUninitialized:false //optional }));</pre>
Set Session Value	<pre>req.session.property = value;</pre>
Destroy Session	<pre>req.session.destroy(err => { ... });</pre>
EJS – View Engine	
Set Views Folder	Structure <pre>project/ ├── app.js ├── views/ │ └── first.ejs</pre>
	<pre>app.set('view engine' , 'ejs')</pre>
	<pre>// ejs file must be in folder named "views" else add below line //app.set('views',path.join(__dirname)); using path module</pre> <pre>app.get("/",(req,res)=>{ res.render("first") })</pre>

Topic	Syntax
	}}

EJS TAGS used in .ejs files

Tag	Description	Example	Output
<% %>	Scriptlet (logic only, no output)	<% if(x>10){ %> OK <% } %>	OK
<%= %>	Output (HTML escaped)	<%= "Hi" %> *Values can be pass	Hi
<%- %>	Output (unescaped HTML)	<%- "Hi" %>	Hi
<%# %>	Comment (not executed)	<%# comment %>	Not visible in browser
<%% %>	Print literal <%	<%% Hello %>	<% Hello %>

Multer – File Upload

Must mention **enctype="multipart/form-data"** in form.html

Import multer	const multer = require('multer');
diskStorage Setup	const storage = multer.diskStorage({ destination: 'updocumnet' // Folder name “updocument” where uploaded files are saved filename: (req, file, cb) => cb(null, file.originalname) });
File key names	fieldname :Field name specified in the form. originalname : Name of the file on the user’s computer. size : Size of the file in bytes. destination : The folder to which the file has been saved. filename : The name of the file within the destination.
File Size Limit	limits: { fileSize: 1024 * 1024} (1MB)
Initiate Multer () middleware	var upload = multer({ storage: store }).single('myfile') //single file fetch from req.file var upload = multer({ storage: store }).array('myfile',5)

Topic	Syntax
	// multiple files fetched from req.files accept upto 5 file, also mention multiple attribute in form.html //myfile is field name mention in form <input type= “file” name= “myfile” multiple> //to inset limit var upload = multer({ storage: store, limits: { fileSize: 1024 * 1024 } }).single('myfile')
RestAPI	
Create Router in data file	const router = express.Router(); also export it at the end of code module.exports=router
Define Route	router.get('/users', (req, res) => { ... });
Use in App file	app.use('/api', router);
Route Parameters	/users/:id, access via req.params.id
Nodemailer	
Import Nodemailer	const nodemailer = require('nodemailer');
Create Transporter	const transporter = nodemailer.createTransport({ host: 'smtp.gmail.com' port: 587/465 auth: { user: 'user@email.com', pass: 'App password' } })
Mailoption object	mailoption= { from: “.....”, to: “.....”, cc: “.....”, subject: “.....”, text or html: `.....` }
Initiate .sendmail() method	transporter.sendMail(mailoption, (err,info)=>{.....})

Sample Examples

Get Method Task

Write express js script to perform tasks as asked below.

1. Create one HTML file which contains two number type input fields, one dropdown which contains options like (select, addition, subtraction, multiplication, division) and one submit button.
2. The input fields must contain the value greater than 0 else it will give a message "Please enter the valid number". Also, user must select any of the formula from the dropdown else give a message "You have not selected any formula". (Message will be displayed on "/calc" page.)
3. If one formula is selected and numbers are entered then respective calculations will be performed on the page "/calc".
4. Use get method to request data.

calc.js

```
var express = require("express");
var app = express();
app.use(express.static("../public", {index: 'calc.html'}));
app.get("/calc", (req, res) => {
  res.set("content-type", "text/html");
  var n1 = parseInt(req.query.n1);
  var n2 = parseInt(req.query.n2);
  if ((n1 > 0) && (n2 > 0)) {
    if (req.query.formula === "addition") {
      a = n1 + n2;
      res.write("<h1>Addition is : " + a + "</h1>");
    }
    else if (req.query.formula === "subtraction") {
      s = n1 - n2;
      res.write("<h1>Subtraction is : " + s + "</h1>");
    }
    else if (req.query.formula === "multi") {
      m = n1 * n2;
      res.write("<h1>Multiplication is : " + m + "</h1>");
    }
    else if (req.query.formula === "div") {
      d = n1 / n2;
      res.write("<h1>Division is : " + d + "</h1>");
    }
    else {
      res.write("<h1>You have not selected any formula.</h1>");
    }
  } else {
    res.write("<h1>Please enter the valid number/s.</h1>");
  }
}
```



```

    res.send()
  })
  app.listen(6012);

```

calc.html

```

<html>
<body>
  <form action="/calc" method="get">
    <fieldset>
      <legend>Calculator:</legend>
      <input type="number" name="n1"><br><br>
      <input type="number" name="n2"><br><br>
      <select name="formula" >
        <option value="">Select formula</option>
        <option value="addition">Addition</option>
        <option value="subtraction">Subtraction</option>
        <option value="multi">Multiplication</option>
        <option value="div">Division</option>
      </select><br><br>
      <input type="submit" value="Submit">
    </fieldset>
  </form>
</body>
</html>

```

Post Method Task

Write express js script to perform the tasks as asked below.

- 1) Create one HTML file named ljform.html and add one form which contains username, password and submit button. Data should be submitted by HTTP post method.
- 2) Submit button is of black color with white text. (External CSS)
- 3) On home page form should be displayed and while submitting the form, on next page named “/login” if username is admin then it will display “Welcome admin” else display “Please login with Admin name”.

src/3.js

```

var expr = require("express");
var app = expr();

app.use(expr.urlencoded({extended: 'true'}));
app.use(expr.static('./public',{index: 'ljform.html'}));
app.post("/login",(req,res)=>{
  if( req.body.uname == 'admin' ){
    res.write(`<h1 style="color:green">Hey ${req.body.uname},Welcome!</h1><br>`)
  }
})

```

```

    }
    else{
      res.write(`<h1 style="color:red">Please login with Admin name</h1>`);
    }
    res.send();
  })
  app.listen(5222);

```

public/ ljform.html

```

<html>
  <head><title>Post Method</title>
    <link rel="stylesheet" href="1.css">
  </head>
  <body>
    <form method="post" action="/login">
      <div class="lj-div">
        <label>User name:</label>
        <input type = "text" name="uname" ></input>
      </div>
      <div class="lj-div">
        <label>Password:</label>
        <input type = "text" name="password" ></input>
      </div>
      <button name="Submit" value="Submit">Submit</button>
    </form>
  </body>
</html>

```

CSS file is also in public folder

```

button{
  padding: 10px;
  background-color: black;
  color: #fff;}

```

Cookie Task

You have been assigned to develop a user feedback form for a website using Express.js and cookies.

Implement the following requirements:

- Create a form with the following fields:
 - Name (input field)
 - Email (input field)
 - Message (textarea field)
 - Rating (radio buttons: Bad, Average, Good, Very Good, Excellent)
- When the user submits the form, store their feedback information (name, email, message, and rating) in a cookie named "feedback" that expires in 10 seconds.

- Display a confirmation message to the user after successfully submitting the form & Create a link to display the feedback details stored in the "feedback" cookie.
- When the user clicks to the link, retrieve the feedback information from the cookie and display it on the page also include a link on the feedback details page to Logout. When the user clicks the link, user redirected to home page.
- After, 10 seconds it will give message “No Feedback available” message to user

feedback.js

```
const express = require('express');
const cp = require('cookie-parser');
const app = express();
app.use(cp());
app.use(express.urlencoded());
app.use(express.static("../public",{index:'feedback.html'}))

app.post('/submit-feedback', (req, res) => {
  const { name, email, message, rating } = req.body;
  // Create the feedback object
  const feedback = {name,email,message,rating};
  // Set the feedback cookie with a 10-second expiration
  res.cookie('feedback', feedback, { maxAge: 10000 });
  res.send('Thank you for your feedback! <br> <a href="/feedback-details"> Show Feedback </a>');
});

app.get('/feedback-details', (req, res) => {
  const feedback = req.cookies.feedback;

  if (feedback) {
    res.send(`
    <h1>Feedback Details</h1>
    <p><strong>Name:</strong> ${feedback.name}</p>
    <p><strong>Email:</strong> ${feedback.email}</p>
    <p><strong>Message:</strong> ${feedback.message}</p>
    <p><strong>Rating:</strong> ${feedback.rating}</p>
    <a href="/" > logout </a>`);
  } else {
    res.send('No feedback available.');
```

feedback.html

```
<body>
<h1>User Feedback Form</h1>
<form action="/submit-feedback" method="POST">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name" required><br><br>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required><br><br>

  <label for="message">Message:</label>
  <textarea id="message" name="message" required></textarea><br><br>

  <label for="rating">Rating:</label>
  <input type="radio" id="rating1" name="rating" value="Bad" required>
  <label for="rating1">Bad</label>
  <input type="radio" id="rating2" name="rating" value="Average" required>
  <label for="rating2">Average</label>
  <input type="radio" id="rating3" name="rating" value="Good" required>
  <label for="rating3">Good</label>
  <input type="radio" id="rating4" name="rating" value="Very Good" required>
  <label for="rating4">Very Good</label>
  <input type="radio" id="rating5" name="rating" value="Excellent" required>
  <label for="rating5">Excellent</label><br><br>
  <input type="submit" value="Submit">
</form>
</body>
```

Session Task

Write a script to meet following requirements:

- Create session.html file page which contains form (username, password, login button). and open it on localhost.
- After clicking submit button, it should jump on “save” page. Store username and password in session.
- After saving session, redirect to “fetchdata” page and read value. On this page check authentication of user. User name and password must be “admin” and “admin@123” respectively.
 - If this condition is true then display welcome admin and display logout link on this page(fetchdata).
- By clicking on logout link user should jump to “destroy” page and destroy the session there and display the message “Session destroyed”. And give the link of “login” under that message. By clicking that link user will be redirected to the home page.
 - Else display “Please enter valid username and password” and login link on this page(fetchdata).

session.html

```
<form method="get" action="/save">
  <div><input type = "text" name="uname" placeholder="Username" ></input></div>
  <div><input type = "password" name="pass" placeholder="Password"> </input>
</div>
  <input type="submit" name="submit" value="Login">
</form>
```

Session.js

```
var express = require("express");
var app = express();
var es = require("express-session");
app.use(es({
  resave:false,
  saveUninitialized:false,
  secret:"ljul"
}));
app.use(express.static(__dirname,{index:'session.html'}));

app.get("/save",(req,res)=>{
  req.session.uname = req.query.uname;
  req.session.password = req.query.pass;
  res.redirect("/fetchdata")
})
app.get("/fetchdata",(req,res)=>{
  if(req.session.uname === "admin" && req.session.password=="admin@123"){
    res.write("<h1 style='color:green;'>Welcome Admin</h1>")
    res.write("<a href='/destroy'>Logout</a>")
  } else {
    res.write("<h1 style='color:red;'>Please enter valid username or password</h1><a href='/'>Login</a>")
  }
  res.send();
});
app.get("/destroy",(req,res)=>{
  req.session.destroy()
  res.write("<h1>Session Destroyed</h1><a href='/'>Login</a>")
  res.send();
});
app.listen(6177);
```

Multer Task

Write an express js script to configure the **multer** middleware. Perform following tasks.

- 1) Create a **html** file named "file.html". This file contains heading(h3) "Upload your CV" in red color. And, a form with input type file(to browse and select file) and submit(to upload the file).
- 2) Create a js file named "file.js" and link this js and html file to browse html file on "/home" page.
- 3) After uploading a file display message on "/upload" page "(file original name) has been uploaded".
- 4) Save uploaded files to specific directory named "upload". And in this folder file must be stored in format of "lju-file.pdf" where "lju" is the field name.

file.html [both files in same folder]

```
<body>
  <h3 style="color:red">Upload your CV</h3>
  <form action="/upload" method="post" enctype="multipart/form-data">
    <input type="file" name="lju">
    <input type="submit" value="Submit">
  </form>
</body>
```

file.js

```
// require the installed packages
const express = require('express')
const multer = require('multer');
const app = express();
app.use(express.static(__dirname, {index: "file.html"}));
// SET STORAGE
var storage = multer.diskStorage({
  destination: "upload",
  filename: function (req, file, cb) {
    cb(null, file.fieldname+"-file.pdf")
  }
})
var upload = multer({ storage: storage })
app.post('/upload', upload.single('lju'), (req, res) => {
  const file = req.file
  console.log(file)
  if (file) {
    res.send("<h1>File <span style='color:red>'"+ file.originalname + "'</span> has been uploaded
in <span style='color:red>' + file.destination + " </span>folder")
  }
})
app.listen(6788);
```

Nodemailer Task

Perform the following tasks as asked.

- Create a HTML file for feedback form and this file should be loaded on home('/') page.
- Fields are :
name, email, dropdown for the feedback ,text area for the additional comments, and submit button.
- Once feedback submitted, message “Thank you for your feedback.” Will be displayed on page '/feedback' and also send mail to the entered email id with the submitted feedback data.
(Data can be submitted using get/post method)

Src/mail_form.js

```
const expr = require("express");
const app = expr();
const nm = require("nodemailer");

// static folder
app.use(expr.static("../public", { index: "mail.html" }));

app.get("/feedback", (req, res) => {
  // store query values
  const { mail, uname, feedback, comments } = req.query;

  // transporter
  const trans = nm.createTransport({
    host: "smtp.gmail.com",
    port: 465,
    secure: true,
    auth: {
      user: "sender@gmail.com",
      pass: "2stepverificationkey"
    }
  });

  // mail options using template literal
  const mailoption = {
    from: "noreply@gmail.com <noreply>",
    to: mail,
    bcc: "bcc@gmail.com",
    subject: "Feedback response",
    html: `
    <h1>Hello ${uname}!</h1>
    <h2> Thank you for submitting the feedback.<br>
    Your feedback:
    <span style="color:red">${feedback}</span>`
  };
});
```

```
    </h2>
    <h3>Additional comments: ${comments}</h3> `
  };

  // send mail
  trans.sendMail(mailoption, (err, info) => {
    if (err) return console.error(err);
    console.log(info);
  });
  res.send("Thank you for your feedback.");
});
app.listen(5001);
```

/public/mail.html

```
<html>
<head><title>Feedback Form</title></head>
<body>
  <form action="/feedback" method="get">
    Name: <input type="text" name="uname"/>
    Email: <input type="text" name="mail"/>
    Feedback: <select name="feedback">
      <option value="bad">Bad</option>
      <option value="good">Good</option>
      <option value="verygood">Very Good</option>
      <option value="excellent">Excellent</option>
    </select>
    Comments: <textarea name="comments" rows="10" cols="15"
placeholder="Comments"></textarea>
    <input type="submit" value="Submit"/>
  </form>
</body>
</html>
```


EJS Task

Develop a web application using Express and EJS to accept marks of four tests (T1, T2, T3, T4), each out of 25, through a form using the POST method. After submission, display all entered marks in a tabular format along with the total marks, and determine the result: if the total is **35 or more**, display **“Pass” in green color**; otherwise, display **“Fail” in red color**. (result also by ejs file)

```
project/
├── app.js
├── views/
│   ├── form.ejs
│   └── result.ejs
```

views/form.ejs

```
<h2>Enter Marks (Each out of 25)</h2>

<form action="/submit" method="POST">
  T1: <input type="number" name="t1" required><br><br>
  T2: <input type="number" name="t2" required><br><br>
  T3: <input type="number" name="t3" required><br><br>
  T4: <input type="number" name="t4" required><br><br>
  <input type="submit" value="Submit">
</form>
```

views/result.ejs

```
<h2>Marks Details</h2>
<table border="1" cellpadding="10">
  <tr>
    <th>T1</th>
    <th>T2</th>
    <th>T3</th>
    <th>T4</th>
  </tr>

  <tr>
    <td><%= t1 %></td>
    <td><%= t2 %></td>
    <td><%= t3 %></td>
    <td><%= t4 %></td>
  </tr>
</table>
<br>
<h2>Total: <%= total %></h2>
<% if (total >= 35) { %>
  <h3 style="color:green;">Pass</h3>
<% } else { %>
  <h3 style="color:red;">Fail</h3>
<% } %>
```

app.js

```
const express = require('express');
const app = express();
app.set('view engine', 'ejs');

// middleware for POST data
app.use(express.urlencoded({ extended: true }));

// show form
app.get('/', (req, res) => {
  res.render('form');
});

// handle form
app.post('/submit', (req, res) => {
  const t1 = parseInt(req.body.t1);
  const t2 = parseInt(req.body.t2);
  const t3 = parseInt(req.body.t3);
  const t4 = parseInt(req.body.t4);
  const total = t1 + t2 + t3 + t4;
  res.render('result', { t1, t2, t3, t4, total });
});
app.listen(3000);
```